Praktikum 9
TensorFlow 2

NAMA	:	Ihsanta Zaki Sanjaya
KELAS	:	2 D4 Teknik Komputer B
NRP	:	3223600058
DOSEN	:	Riyanto Sigit, ST, M.Kom, Ph.D
MATA KULIAH	:	Praktikum Sistem Cerdas
TANGGAL	:	Rabu, 14 Mei 2025

D4 TEKNIK KOMPUTER
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
2025

Praktikum 9

TensorFlow 2

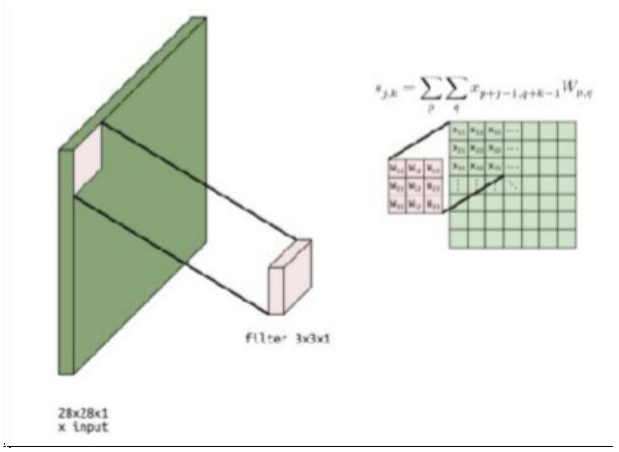
A. Tujuan

1. Mahasiswa dapat memahami dan menjelaskan konsep TensorFlow.
2. Mahasiswa dapat menjelaskan model TensorFlow.
3. Mahasiswa dapat membuat aplikasi TensorFlow.

B. Pendahuluan

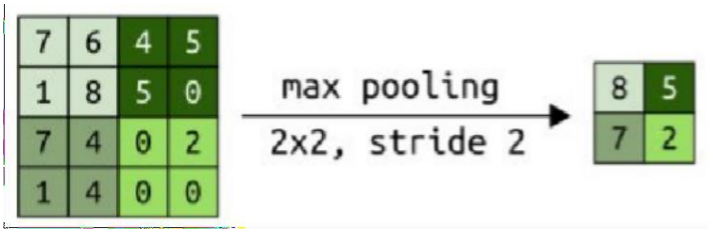
b. Klasifikasi Gambar Fashion dengan CNN

Convolutional Neural Network (CNN) adalah jenis arsitektur jaringan saraf yang umum digunakan untuk pengenalan gambar. Pada bagian ini, kita akan menggunakan CNN untuk mengklasifikasikan kategori fashion dari dataset Fashion MNIST.



Konvolusi

Lapisan utama dalam CNN adalah lapisan konvolusi. Proses konvolusi bertujuan untuk mengekstrak fitur-fitur penting dari gambar, seperti tepi (edges), sudut (corners), dan pola tertentu. Operasi konvolusi dilakukan dengan menggeser filter (kernel) kecil di atas gambar. Hasil dari setiap pergeseran berupa satu nilai yang bisa diibaratkan sebagai aktivasi dari satu neuron. Nilai-nilai dalam filter tersebut akan dipelajari secara otomatis selama proses pelatihan.



Max Pooling

Setelah lapisan konvolusi, biasanya diikuti oleh lapisan max pooling. Fungsi dari max pooling adalah untuk mereduksi dimensi data dan memilih fitur yang paling menonjol (teraktivasi). Sebagai contoh, pada pooling 2x2 dengan stride 2, kita mengambil nilai maksimum dari setiap area 2x2 pada output konvolusi, lalu menggeser filter sejauh dua langkah ke kanan atau ke bawah.

Pooling ini membantu model dalam menjaga informasi penting sekaligus mengurangi kompleksitas komputasi. Dengan demikian, CNN dapat lebih fokus pada fitur-fitur utama saat mempelajari pola dari gambar.

c. Klasifikasi Ekspresi Emosi dengan CNN

Pada bagian ini, kita akan menggunakan model CNN untuk mengklasifikasikan ekspresi emosi dari gambar, yaitu antara ekspresi senang (happy) dan sedih (sad). Dataset disiapkan oleh Laurence Moroney (AI Lead di Google) dan dapat diunduh melalui tautan berikut:

https://storage.googleapis.com/laurencemoroney-blog.appspot.com/happy-or-sad.zip

Dataset ini terdiri dari 80 gambar, masing-masing 40 gambar untuk ekspresi senang dan sedih. Semua gambar dibuat menggunakan karakter emoji 3D.

Pemrosesan Data

Dalam contoh ini, kita menggunakan kelas ImageDataGenerator dari TensorFlow untuk melakukan preprocessing data gambar. Kelas ini akan dibahas lebih lanjut pada bagian berikutnya. Kita juga akan memuat gambar langsung dari direktori, bukan dengan membaca semua gambar sekaligus ke dalam memori program.

Untuk keperluan pengujian, kita bisa menggunakan gambar lain yang memiliki ekspresi senang atau sedih, atau mengunduh empat gambar uji dari tautan berikut:

https://bit.ly/36LmNZe

Overfitting pada Model

Contoh ini juga menunjukkan bahwa jika jumlah data pelatihan sangat sedikit dan kita menggunakan gambar baru (yang belum pernah dilihat model) untuk pengujian, maka hasil prediksi cenderung buruk. Menambahkan lebih banyak lapisan ke model justru bisa memperparah kondisi ini, karena model menjadi overfitting, yaitu terlalu menyesuaikan diri dengan data pelatihan dan tidak mampu menggeneralisasi ke data baru. Overfitting merupakan salah satu masalah umum dalam machine learning yang perlu dihindari.

3. Convolutional Neural Network (CNN) di TensorFlow

a. Klasifikasi Gambar Kucing dan Anjing

Dalam bagian ini, kita akan membangun model CNN untuk mengklasifikasikan gambar kucing dan anjing. Dataset yang digunakan berasal dari kompetisi *Kaggle Dogs*

vs. Cats, namun untuk keperluan praktikum, kita akan menggunakan versi ringkas (sekitar 1/10 dari total data).

Dataset versi kecil ini dapat diunduh melalui tautan berikut:

https://storage.googleapis.com/mledu-datasets/cats_and_dogs_filtered.zip

Dataset ini telah difilter dan dipisahkan ke dalam folder pelatihan dan validasi untuk masing-masing kelas: kucing dan anjing.

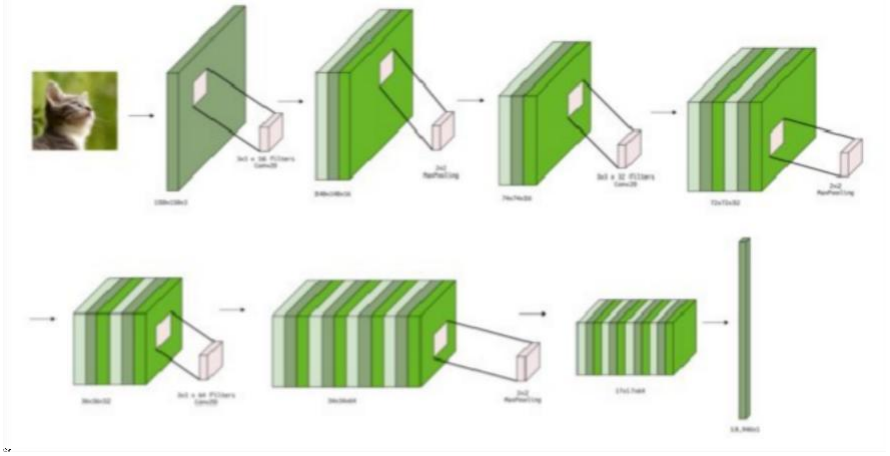
Untuk pengujian model, dapat menggunakan 2 gambar kucing dan 2 gambar anjing yang tersedia di tautan berikut:

https://bit.ly/3xYRjuw

Dengan dataset ini, kita dapat mulai mengembangkan model klasifikasi gambar menggunakan TensorFlow dan mengamati bagaimana CNN belajar membedakan antara dua jenis objek berdasarkan fitur visualnya.

Contoh ini juga menunjukkan bahwa jika jumlah data pelatihan sangat sedikit dan kita menggunakan gambar baru (yang belum pernah dilihat model) untuk pengujian, maka hasil prediksi cenderung buruk. Menambahkan lebih banyak lapisan ke model justru bisa memperparah kondisi ini, karena model menjadi overfitting, yaitu terlalu menyesuaikan diri dengan data pelatihan dan tidak mampu menggeneralisasi ke data baru. Overfitting merupakan salah satu masalah umum dalam machine learning yang perlu dihindari.

Di bawah ini ditunjukkan bagaimana tensor dari citra input (array multi-dimensi) mengalami perubahan dimensi setelah melewati setiap operasi di masing-masing lapisan.



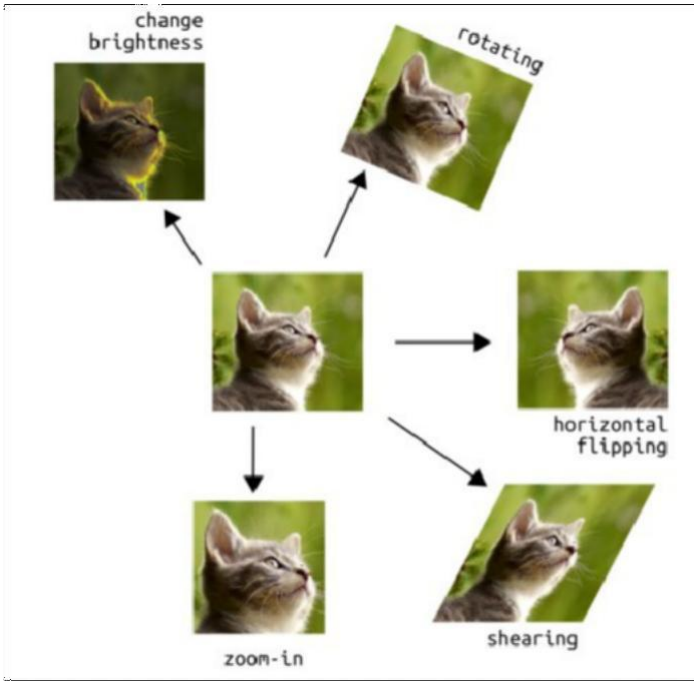
b. Meningkatkan Akurasi Klasifikasi Kucing dan Anjing

Jika kita menjalankan kode Listing 3.1, terlihat bahwa akurasi pada data pelatihan mendekati 1, namun akurasi pada data validasi hanya mencapai sekitar 0,72. Hal ini menunjukkan terjadinya overfitting, yaitu kondisi saat model belajar terlalu baik pada data pelatihan namun gagal melakukan generalisasi ke data baru.

Untuk mengatasi overfitting, terdapat beberapa teknik yang umum digunakan. Dalam kasus ini, kita akan menggunakan dua metode: augmentasi citra (image augmentation) dan dropout layer.

Image Augmentation

Image augmentation adalah teknik untuk memperbanyak variasi data pelatihan dengan melakukan transformasi pada gambar yang sudah ada. Transformasi ini bisa berupa: Zoom in / Zoom out, Rotasi, Shearing, Flipping, Shifting



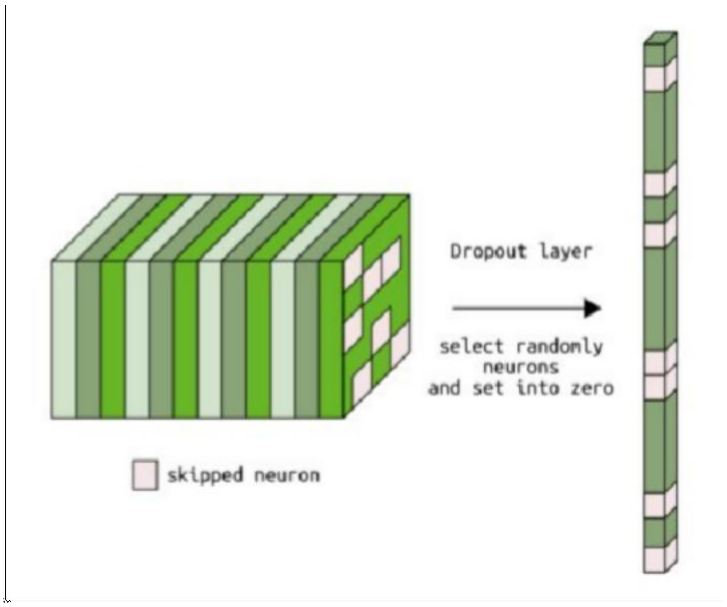
TensorFlow menyediakan sebuah kelas yang sangat berguna untuk melakukan berbagai transformasi citra yang telah dijelaskan sebelumnya, yaitu ImageDataGenerator(). Kelas ini telah diperkenalkan sebelumnya pada Listing 2.5. Pada kode selanjutnya, kita akan menggunakan beberapa atribut dan metode tambahan dari kelas ini untuk melakukan image augmentation.

Dropout Layer

Sementara itu, dropout layer berfungsi sebagai lapisan masking yang secara acak "melewati" (menonaktifkan) sebagian unit input selama proses pelatihan. Dalam praktiknya, dropout biasanya ditempatkan sebelum lapisan fully-connected (dense) dan setelah seluruh kombinasi lapisan konvolusi dan max pooling.

Tujuannya adalah agar model terlebih dahulu mempelajari semua kemungkinan fitur dari proses konvolusi dan pooling. Setelah itu, dengan menonaktifkan beberapa neuron melalui dropout, model diharapkan hanya mempertahankan fitur-fitur paling penting dan mengabaikan fitur yang tidak relevan atau berulang (redundan).

Ilustrasi proses komputasi pada dropout layer ditunjukkan pada gambar di bawah ini.

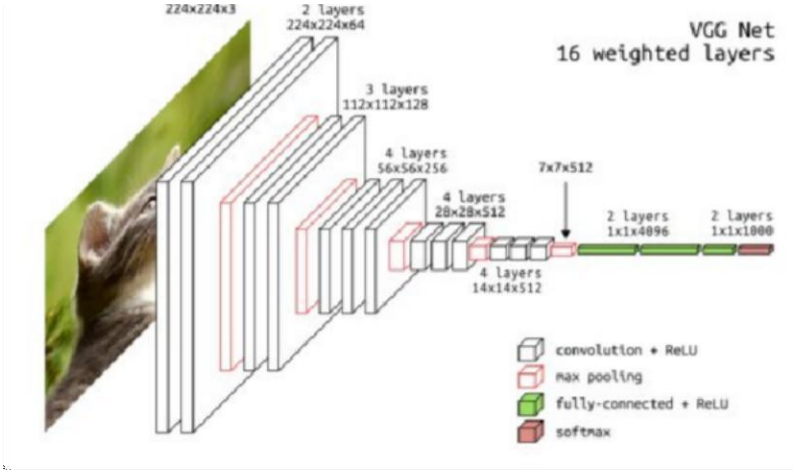


c. Transfer Learning dengan VGG Net

Sekarang, kita akan menggunakan teknik yang sangat berguna dalam machine learning, yaitu transfer learning, di mana kita tidak perlu melatih model dari awal dengan menggunakan jumlah data yang besar jika kita hanya memiliki data yang sedikit. Transfer learning memungkinkan kita untuk menggunakan model yang telah dilatih sebelumnya (pre-trained model) dan "membekukan" sebagian besar lapisan, lalu hanya "melepaskan" beberapa lapisan di bagian akhir untuk dilatih menggunakan data kita yang lebih kecil.

Ide dasarnya adalah jika masalah yang kita hadapi serupa dengan masalah yang sudah diselesaikan oleh model yang telah dilatih sebelumnya, maka kita dapat melatih model dengan lebih cepat dan mendapatkan klasifikator yang akurat meskipun dengan data yang terbatas.

Untuk menunjukkan manfaat dari transfer learning, kita akan menggunakannya dengan VGG Net. Untuk deskripsi lebih lanjut tentang arsitektur CNN ini, Anda bisa membaca makalah dari Simonyan dan Zisserman (2014). Berikut adalah gambaran abstrak sederhana dari VGG Net.



d. Transfer Learning dengan VGG Net

Pada bagian ini, kita akan mengakhiri bab ini dengan melakukan klasifikasi menggunakan dataset bahasa isyarat. Berikut adalah tautan untuk mengunduh dataset dan empat gambar contoh:

Data pelatihan: https://bit.ly/3kStktw

Data validasi: https://bit.ly/3kUArl0

Empat gambar contoh: https://bit.ly/3iJFke1

Dataset ini berasal dari kompetisi Kaggle Sign Language MNIST (link ke Kaggle), dan kami memberikan tautan tersebut agar Anda tidak perlu login untuk mengunduh data.

Dataset ini disimpan dalam format .csv, di mana setiap kolom mewakili nilai piksel dalam rentang 0 hingga 255. Setiap gambar memiliki ukuran 28x28 piksel, dan setiap baris mewakili satu gambar isyarat tangan. Berikut adalah tabel yang menunjukkan pasangan gambar dengan huruf alfabet yang sesuai.

Jika memplot salah satu gambar dari file .csv, akan didapatkan gambar dalam versi grayscale dengan resolusi yang lebih kecil. Jika dilihat dengan cermat pada tabel di atas, bahwa tidak ada representasi tanda tangan untuk huruf "J" dan "Z". Kedua huruf ini digambarkan dengan gerakan jari, sehingga tidak dapat disertakan dalam dataset ini.

C. HASIL PERCOBAAN

Fashion_classifier_with_cnn.py

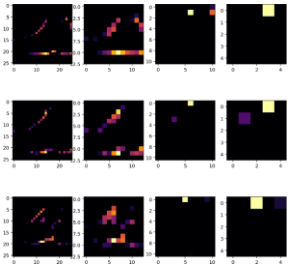
```
import tensorflow as tf
import matplotlib.pyplot as plt
import numpy as np

def create_cnn_model():
    mnist = tf.keras.datasets.fashion_mnist
    (training_images, training_labels), (test_images, test_labels) =
mnist.load_data()
    model = tf.keras.models.Sequential([
        tf.keras.layers.Conv2D(64, (3, 3), activation="relu", input_shape=(28, 28,
1)),
        tf.keras.layers.MaxPooling2D(2, 2),
        tf.keras.layers.Conv2D(64, (3, 3), activation="relu"),
        tf.keras.layers.MaxPooling2D(2, 2),
        tf.keras.layers.Flatten(),
        tf.keras.layers.Dense(128, activation="relu"),
        tf.keras.layers.Dense(10, activation="softmax")
    ])
    model.compile(optimizer="adam", loss="sparse_categorical_crossentropy",
metrics=["accuracy"])
    model.summary()
    model.fit(training_images, training_labels, epochs=10)
    test_loss = model.evaluate(test_images, test_labels)
    return [model, test_loss]

def visualizing_conv_and_max_pool(model):
    mnist = tf.keras.datasets.fashion_mnist
    (_, _), (test_images, test_labels) = mnist.load_data()
    # from this print out, we can decide the indices for
    # first_image, second_image, third_image
    print(test_labels[:100].reshape(10, 10))
    f, ax_arr = plt.subplots(3, 4, figsize=(10, 10))
    first_image = 0
    second_image = 28
    third_image = 23
    convolution_number = 2 # starting from 0 to 63
    layer_outputs = [layer.output for layer in model.layers]
    activation_model = tf.keras.models.Model(inputs=model.input,
outputs=layer_outputs)
    for x in range(0, 4):
        f1 = activation_model.predict(test_images[first_image].reshape(1, 28, 28,
1))[x]
        ax_arr[0, x].imshow(f1[0, :, :, convolution_number], cmap="inferno")
        ax_arr[0, x].grid(False)
        f2 = activation_model.predict(test_images[second_image].reshape(1, 28, 28,
1))[x]
        ax_arr[1, x].imshow(f2[0, :, :, convolution_number], cmap="inferno")
        ax_arr[1, x].grid(False)
```

```
        f3 = activation_model.predict(test_images[third_image].reshape(1, 28, 28,
1))[x]
        ax_arr[2, x].imshow(f3[0, :, :, convolution_number], cmap="inferno")
        ax_arr[2, x].grid(False)
    plt.pause(2)

if __name__ == "__main__":
    cnn_model, cnn_test_loss = create_cnn_model()
    cnn_model.save("model-saved/category2.h5")
    visualizing_conv_and_max_pool(cnn_model)
```

```
Epoch 10/10
1875/1875 [==============================] - 40s 21ms/step - loss: 0.1786 - accuracy: 0.9323
313/313 [==============================] - 2s 7ms/step - loss: 0.3625 - accuracy: 0.8966
```

```
[[9 2 1 1 6 1 4 6 5 7]
 [4 5 7 3 4 1 2 4 8 0]
 [2 5 7 9 1 4 6 0 9 3]
 [8 8 3 3 8 0 7 5 7 9]
 [6 1 3 7 6 7 2 1 2 2]
 [4 4 5 8 2 2 8 4 8 0]
 [7 7 8 5 1 1 2 3 9 8]
 [7 0 2 6 2 3 1 2 8 4]
 [1 8 5 9 5 0 3 2 0 6]
```

```
 [5 3 6 7 1 8 0 1 4 2]]
1/1 [==============================] - 0s 221ms/step
1/1 [==============================] - 0s 33ms/step
1/1 [==============================] - 0s 32ms/step
1/1 [==============================] - 0s 35ms/step
1/1 [==============================] - 0s 32ms/step
1/1 [==============================] - 0s 39ms/step
1/1 [==============================] - 0s 32ms/step
1/1 [==============================] - 0s 31ms/step
1/1 [==============================] - 0s 34ms/step
1/1 [==============================] - 0s 36ms/step
1/1 [==============================] - 0s 31ms/step
1/1 [==============================] - 0s 32ms/step
```



Face expression classifier with cnn

```
import os
import zipfile
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
import matplotlib.image as mpimg

from tensorflow.keras.optimizers import Adam
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.preprocessing import image as keras_image

class MyCallback(tf.keras.callbacks.Callback):
    def __init__(self, desired_accuracy):
        super(MyCallback, self).__init__()
        self.DESIRED_ACCURACY = desired_accuracy
    def on_epoch_end(self, epoch, logs=None):
        if logs.get("accuracy") > self.DESIRED_ACCURACY:
            print(f"\nReached {self.DESIRED_ACCURACY * 100:.2f}" + " accuracy so
cancelling training!")
            self.model.stop_training = True

def load_dataset(zip_file_path, extracted_zip_file_path, train_happy_dir,
train_sad_dir):
    zip_ref = zipfile.ZipFile(zip_file_path, "r")
    zip_ref.extractall(extracted_zip_file_path)
    zip_ref.close()
    train_happy_names = os.listdir(train_happy_dir)
    train_sad_names = os.listdir(train_sad_dir)
    print(train_happy_names[:10])
    print(train_sad_names[:10])
    print(f"total training happy images {len(train_happy_names)}")
    print(f"total training sad images {len(train_sad_names)}")
    return train_happy_names, train_sad_names

def do_data_preprocessing(dataset_dir):
    train_datagen = ImageDataGenerator(rescale=1. / 255)
    train_generator = train_datagen.flow_from_directory(
        dataset_dir,
        target_size=(150, 150),
        batch_size=8, # reduce this from 128 (our dataset is small)
        class_mode="binary"
    )
    return train_generator

def create_cnn_model():
    model = tf.keras.models.Sequential([
        tf.keras.layers.Conv2D(16, (3, 3), activation="relu", input_shape=(150, 150,
3)),
        tf.keras.layers.MaxPooling2D(2, 2),
        tf.keras.layers.Conv2D(32, (3, 3), activation="relu"),
        tf.keras.layers.MaxPooling2D(2, 2),
        tf.keras.layers.Conv2D(64, (3, 3), activation="relu"),
```

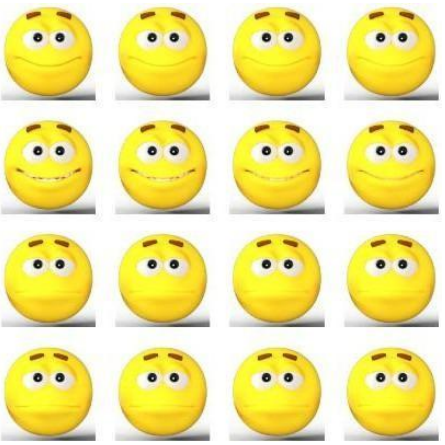
```
        tf.keras.layers.MaxPooling2D(2, 2),
        tf.keras.layers.Flatten(),
        tf.keras.layers.Dense(512, activation="relu"),
        tf.keras.layers.Dense(1, activation="sigmoid")
    ])
    model.compile(loss="binary_crossentropy", optimizer=Adam(learning_rate=0.001),
metrics=["accuracy"])
    model.summary()
    return model

def plot_training_images(train_happy_dir, train_sad_dir, train_happy_names,
train_sad_names):
    # Parameter for our graph; we'll output images in a 4x4 configuration
    n_rows = 4
    n_cols = 4
    # Index for iterating over images
    img_index = 0
    # Set up matplotlib fig, and size it to fit 4x4 pics
    fig = plt.gcf()
    fig.set_size_inches(n_cols * 4, n_rows * 4)
    img_index += 8
    next_happy_img = [os.path.join(train_happy_dir, fname)
                      for fname in train_happy_names[img_index - 8:img_index]]
    next_sad_img = [os.path.join(train_sad_dir, fname)
                      for fname in train_sad_names[img_index - 8:img_index]]
    for i, img_path in enumerate(next_happy_img + next_sad_img):
        # Set up subplot; subplot indices start at 1
        sp = plt.subplot(n_rows, n_cols, i + 1)
        sp.axis("off") # don't show axes (or gridlines)
        img = mpimg.imread(img_path)
        plt.imshow(img)
        plt.pause(1.0)

def classify_images(fn_arr, model):
    for fn in fn_arr:
        path = "datasets/" + fn
        img = keras_image.load_img(path, target_size=(150, 150))
        x = keras_image.img_to_array(img)
        x = np.expand_dims(x, axis=0)
        image_i = np.vstack([x])
        classes = model.predict(image_i, batch_size=10)
        print(classes[0])
        if classes[0] > 0.5:
            print(fn + " is happy")
        else:
            print(fn + " is sad")

if __name__ == "__main__":
    zip_file_path = "datasets/happy-or-sad.zip"
    extracted_zip_file_path = "datasets/happy-or-sad"
    # Directory with our training happy images
    train_happy_dir = os.path.join("datasets/happy-or-sad/happy")
    # Directory with our training sad images
    train_sad_dir = os.path.join("datasets/happy-or-sad/sad")
```

```
    # Extract zip and get path to the data sets
    train_happy_names, train_sad_names = load_dataset(zip_file_path,
extracted_zip_file_path,
    train_happy_dir, train_sad_dir)
    # Plot the data sets
    plot_training_images(train_happy_dir, train_sad_dir, train_happy_names,
train_sad_names)
    # Data preprocessing
    train_generator = do_data_preprocessing(extracted_zip_file_path)
    # Building a small CNN model
    cnn_model = create_cnn_model()
    # Training the model to the training data
    DESIRED_ACCURACY = 0.99
    callbacks = MyCallback(DESIRED_ACCURACY)
    history = cnn_model.fit(
        train_generator,
        steps_per_epoch=8,
        epochs=50,
        verbose=1,
        callbacks=[callbacks]
    )
    cnn_model.save("happy_sad_model.h5")
    #Predict some images
    fn_arr = ["beauty-1132617_640.jpg", "girl-2961959_640.jpg",
              "woman-2126727_640.jpg", "beautiful-18279_640.jpg"]
    classify_images(fn_arr, cnn_model)
```

Hasil

```
Reached 99.00 accuracy so cancelling training!
8/8 [==============================] - 1s 75ms/step - loss: 0.0281 - accuracy: 1.0000
1/1 [==============================] - 0s 139ms/step
```

```
[1.]
beauty-1132617_640.jpg is happy
1/1 [==============================] - 0s 31ms/step
[1.]
girl-2961959_640.jpg is happy
1/1 [==============================] - 0s 30ms/step
[1.]
woman-2126727_640.jpg is happy
1/1 [==============================] - 0s 33ms/step
[1.]
beautiful-18279_640.jpg is happy
```



Cats and dogs classifier with cnn

```
import os
import zipfile
import numpy as np
import random
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
import tensorflow as tf
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.preprocessing.image import ImageDataGenerator, img_to_array,
load_img
from tensorflow.keras.preprocessing import image as keras_image
def load_dataset(zip_file_path, extracted_zip_file_path):
    zip_ref = zipfile.ZipFile(zip_file_path, "r")
    zip_ref.extractall(os.path.split(extracted_zip_file_path)[0])
    zip_ref.close()
    train_dir = os.path.join(extracted_zip_file_path, "train")
    validation_dir = os.path.join(extracted_zip_file_path, "validation")
    train_cats_dir = os.path.join(train_dir, "cats")
    train_dogs_dir = os.path.join(train_dir, "dogs")
    validation_cats_dir = os.path.join(validation_dir, "cats")
    validation_dogs_dir = os.path.join(validation_dir, "dogs")
    train_cat_fnames = os.listdir(train_cats_dir)
    train_dog_fnames = os.listdir(train_dogs_dir)
    validation_cat_fnames = os.listdir(validation_cats_dir)
    validation_dog_fnames = os.listdir(validation_dogs_dir)
    print(train_cat_fnames[:10])
    print(train_dog_fnames[:10])
    print("total training cat images :", len(train_cat_fnames))
    print("total training dog images :", len(train_dog_fnames))
    print("total validation cat images:", len(validation_cat_fnames))
    print("total validation dog images:", len(validation_dog_fnames))
    return train_dir, validation_dir, train_cats_dir, train_dogs_dir, \
    train_cat_fnames, train_dog_fnames

def do_data_preprocessing(train_dir, validation_dir):
    train_datagen = ImageDataGenerator(rescale=1. / 255)
    validation_datagen = ImageDataGenerator(rescale=1. / 255)
    train_generator = train_datagen.flow_from_directory(
    train_dir,
    batch_size=20,
    class_mode="binary",
    target_size=(150, 150)
    )
    validation_generator = validation_datagen.flow_from_directory(
    validation_dir,
    batch_size=20,
    class_mode="binary",
    target_size=(150, 150)
```

```
    )
    return train_generator, validation_generator

def create_cnn_model():
    # Building a small model from scratch: validation accuracy ~ 72 %
    model = tf.keras.models.Sequential([
    tf.keras.layers.Conv2D(16, (3, 3), activation="relu", input_shape=(150, 150,
3)),
    tf.keras.layers.MaxPooling2D(2, 2),
    tf.keras.layers.Conv2D(32, (3, 3), activation="relu"),
    tf.keras.layers.MaxPooling2D(2, 2),
    tf.keras.layers.Conv2D(64, (3, 3), activation="relu"),
    tf.keras.layers.MaxPooling2D(2, 2),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(512, activation="relu"),
    tf.keras.layers.Dense(1, activation="sigmoid")
    ])
    model.summary()
    model.compile(optimizer=Adam(learning_rate=0.001),
    loss="binary_crossentropy",
    metrics=["accuracy"])
    return model

def plot_cats_and_dogs(train_cats_dir, train_dogs_dir,
train_cat_fnames, train_dog_fnames):
    # Parameter for our graph; we'll output images in a 4x4 configuration
    nrows = 4
    ncols = 4
    # Index for iterating over images
    pic_index = 0
    # Set up matplotlib fig, and size it to fit 4x4 pics
    fig = plt.gcf()
    fig.set_size_inches(ncols * 4, nrows * 4)
    pic_index += 8
    next_cat_pix = [os.path.join(train_cats_dir, fname)
    for fname in train_cat_fnames[pic_index - 8:pic_index]]
    next_dog_pix = [os.path.join(train_dogs_dir, fname)
    for fname in train_dog_fnames[pic_index - 8:pic_index]]
    for i, img_path in enumerate(next_cat_pix + next_dog_pix):
        # Set up subplot; subplot indices start at 1
        sp = plt.subplot(nrows, ncols, i + 1)
        sp.axis("off") # Don't show axes (or gridlines)
        img = mpimg.imread(img_path)
        plt.imshow(img)
        plt.pause(1.0)

def classify_images(fn_arr, model):
    for fn in fn_arr:
        path = "datasets/" + fn
        img = keras_image.load_img(path, target_size=(150, 150))
        x = keras_image.img_to_array(img)
```

```
        x = np.expand_dims(x, axis=0)
        image_i = np.vstack([x])
        classes = model.predict(image_i, batch_size=10)
        print(classes[0])
        if classes[0] > 0.5:
            print(fn + " is a dog")
        else:
            print(fn + " is a cat")

def plot_intermediate_repr(model, train_cats_dir, train_dogs_dir,
train_cat_fnames, train_dog_fnames):
    # Let's define a new Model that will take an image as input, and will output
    # intermediate representations for all layers in the previous model after the
first.
    successive_outputs = [layer.output for layer in model.layers[0:]]
    visualization_model = tf.keras.models.Model(inputs=model.input,
    outputs=successive_outputs)
    # Let's prepare a random input image from the training set.
    cat_img_files = [os.path.join(train_cats_dir, f) for f in train_cat_fnames]
    dog_img_files = [os.path.join(train_dogs_dir, f) for f in train_dog_fnames]
    img_path = random.choice(cat_img_files + dog_img_files)
    img = load_img(img_path, target_size=(150, 150)) # this is a PIL image
    x = img_to_array(img) # numpy array with shape (150, 150, 3)
    print(f"x.shape : {x.shape}")
    x = x.reshape((1,) + x.shape)
    x = x / 255.
    # Let's run our image through our network, thus obtaining all
    # intermediate representations for this image.
    successive_feature_maps = visualization_model.predict(x)
    # These are the names of the layers, so can have them as part of our plot
    layer_names = [layer.name for layer in model.layers]
    # Now let's display our representations
    for layer_name, feature_map in zip(layer_names, successive_feature_maps):
        # Just do this for the conv / maxpool layers, not the fully connected layers
        if len(feature_map.shape) == 4:
            n_features = feature_map.shape[-1] # number of features (filters) in
feature map
            # The feature map has shape (1, size, size, n_features)
            size = feature_map.shape[1]
            # We will tile our images in this matrix
            display_grid = np.zeros((size, size * n_features))
            for i in range(n_features):
                # Post-process the feature to make it visually palatable
                x = feature_map[0, :, :, i]
                x -= x.mean()
                # print('x.std()', x.std())
                x = x / x.std() if x.std() > 1e-14 else x
                x *= 64
                x += 128
                x = np.clip(x, 0, 255).astype("uint8")
                # We will tile each filter into this big horizontal grid
```

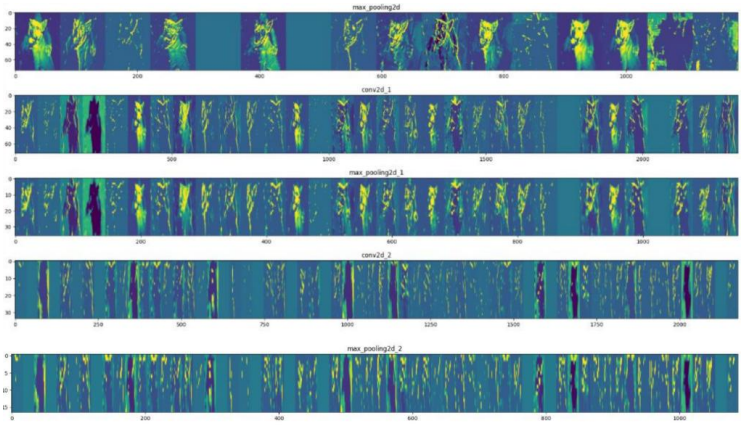
```
                display_grid[:, i * size: (i + 1) * size] = x
                # Display the grid
            plt.figure(figsize=(20, 2))
            plt.title(layer_name)
            plt.grid(False)
            plt.imshow(display_grid, aspect="auto", cmap="viridis")
            plt.subplots_adjust(left=0.03, right=0.99)
    plt.pause(1.0)

def plot_history(train, val, title):
    epochs = range(len(train))
    plt.figure()
    plt.plot(epochs, train, label="train")
    plt.plot(epochs, val, label="val")
    plt.title(title)
    plt.legend(loc="best")
    plt.pause(1.0)

if __name__ == "__main__":
    zip_file_path = "datasets/cats_and_dogs_filtered.zip"
    extracted_zip_file_path = "datasets/cats_and_dogs_filtered"
    # Extract zip and get path to the data sets
    train_dir, validation_dir, train_cats_dir, train_dogs_dir, \
    train_cat_fnames, train_dog_fnames \
    = load_dataset(zip_file_path, extracted_zip_file_path)
    # Plot the data sets
    plot_cats_and_dogs(train_cats_dir, train_dogs_dir,
    train_cat_fnames, train_dog_fnames)
    # Data preprocessing
    train_generator, validation_generator \
    = do_data_preprocessing(train_dir, validation_dir)
    # Building a small CNN model
    cnn_model = create_cnn_model()
    history = cnn_model.fit(
    train_generator,
    validation_data=validation_generator,
    steps_per_epoch=100,
    epochs=15,
    validation_steps=50,
    verbose=1
    )
    fn_arr = ["cat-2083492_only_head.jpg", "cat-1146504_640.jpg",
    "dog-3846767_640.jpg", "dog-3388069_640.jpg"]
    classify_images(fn_arr, cnn_model)
    # Visualizing intermediate representations
    plot_intermediate_repr(cnn_model, train_cats_dir, train_dogs_dir,
    train_cat_fnames, train_dog_fnames)
    # Evaluating accuracy and loss for the model
    acc = history.history["accuracy"]
    val_acc = history.history["val_accuracy"]
    loss = history.history["loss"]
```

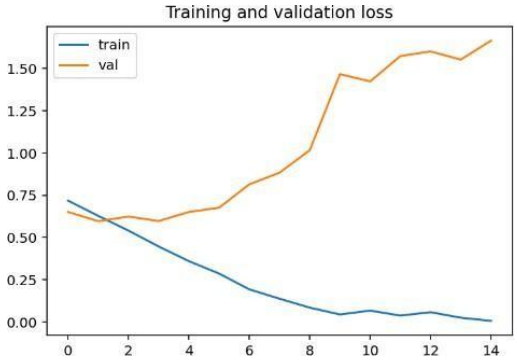
```
    val_loss = history.history["val_loss"]
    plot_history(acc, val_acc, "Training and validation accuracy")
    plot_history(loss, val_loss, "Training and validation loss")
```

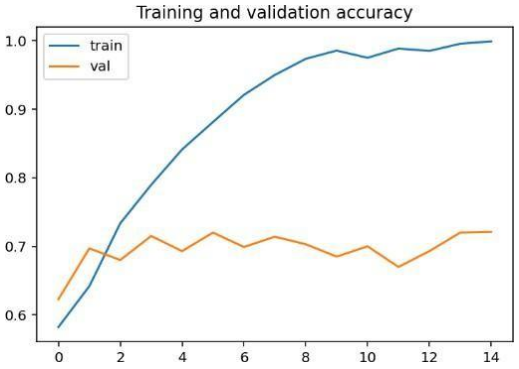
Hasil

```
Epoch 15/15
100/100 [==============================] - 18s 176ms/step - loss: 0.0058 - accuracy: 0.9985 - val_loss: 1.6654 - val_accuracy: 0.721
1/1 [==============================] - 0s 114ms/step
[1.]
cat-2083492_only_head.jpg is a dog
1/1 [==============================] - 0s 28ms/step
[1.]
```

```
cat-1146504_640.jpg is a dog
1/1 [==============================] - 0s 29ms/step
[1.]
dog-3846767_640.jpg is a dog
1/1 [==============================] - 0s 30ms/step
[1.]
dog-3388069_640.jpg is a dog
x.shape : (150, 150, 3)
1/1 [==============================] - 0s 135ms/step
```







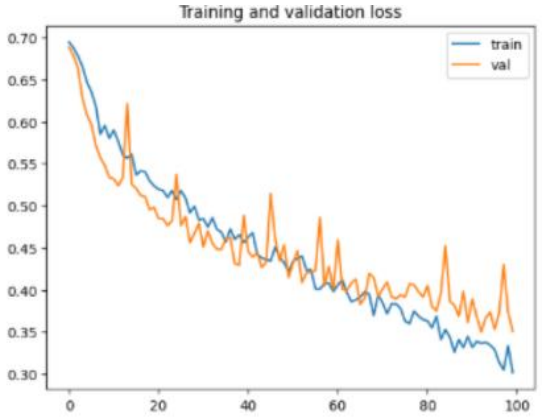
Cats and dogs classifier with imagedatgen and dropout

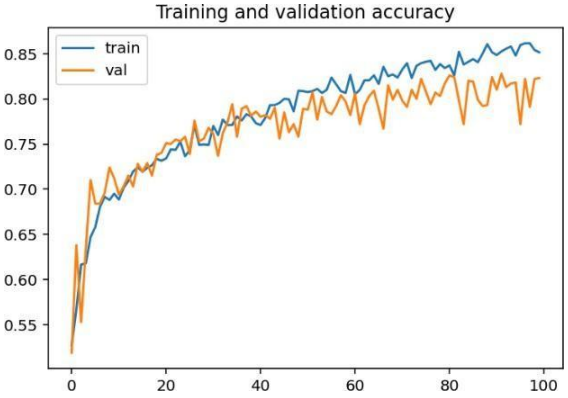
```
import os
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.preprocessing import image as keras_image

def plot_history(train, val, title):
    epochs = range(len(train))
    plt.figure()
    plt.plot(epochs, train, label="train")
    plt.plot(epochs, val, label="val")
    plt.title(title)
    plt.legend(loc="best")
    plt.pause(1.0)

def do_data_preprocessing(train_dir, validation_dir, aug=False):
    if aug:
        train_datagen = ImageDataGenerator(
        rescale=1./255,
        rotation_range=40,
        width_shift_range=0.2,
        height_shift_range=0.2,
        shear_range=0.2,
        zoom_range=0.2,
        horizontal_flip=True,
        fill_mode="nearest"
        )
    else:
        train_datagen = ImageDataGenerator(rescale=1./255)
    validation_datagen = ImageDataGenerator(rescale=1./255)
    train_generator = train_datagen.flow_from_directory(
        train_dir,
        target_size=(150, 150),
        batch_size=20,
        class_mode="binary"
        )
    validation_generator = validation_datagen.flow_from_directory(
        validation_dir,
        target_size=(150, 150),
        batch_size=20,
        class_mode="binary"
        )
    return train_generator, validation_generator

def create_cnn_model():
    model = tf.keras.models.Sequential([
    tf.keras.layers.Conv2D(32, (3, 3), activation="relu", input_shape=(150, 150,
3)),
    tf.keras.layers.MaxPooling2D(2, 2),
```

```
    tf.keras.layers.Conv2D(64, (3, 3), activation="relu"),
    tf.keras.layers.MaxPooling2D(2, 2),
    tf.keras.layers.Conv2D(128, (3, 3), activation="relu"),
    tf.keras.layers.MaxPooling2D(2, 2),
    tf.keras.layers.Conv2D(128, (3, 3), activation="relu"),
    tf.keras.layers.MaxPooling2D(2, 2),
    tf.keras.layers.Dropout(0.5), # 3rd options (with image augmentation)
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(512, activation="relu"),
    tf.keras.layers.Dense(1, activation="sigmoid")
    ])
    model.compile(loss="binary_crossentropy",
    optimizer=Adam(learning_rate=1e-4),
    metrics=["accuracy"])
    model.summary()
    return model

def classify_images(fn_arr, model):
    for fn in fn_arr:
        path = "datasets/" + fn
        img = keras_image.load_img(path, target_size=(150, 150))
        x = keras_image.img_to_array(img)
        x = np.expand_dims(x, axis=0)
        image_i = np.vstack([x])
        classes = model.predict(image_i, batch_size=10)
        print(classes[0])
        if classes[0] > 0.5:
            print(fn + " is a dog")
        else:
            print(fn + " is a cat")

if __name__== "__main__":
    base_dir = "datasets/cats_and_dogs_filtered"
    train_dir = os.path.join(base_dir, "train")
    validation_dir = os.path.join(base_dir, "validation")
    train_cats_dir = os.path.join(train_dir, "cats")
    train_dogs_dir = os.path.join(train_dir, "dogs")
    validation_cats_dir = os.path.join(train_dir, "dogs")
    validation_dogs_dir = os.path.join(validation_dir, "cats")
    print(len(os.listdir(train_cats_dir)))
    print(len(os.listdir(train_dogs_dir)))
    print(len(os.listdir(validation_cats_dir)))
    print(len(os.listdir(validation_dogs_dir)))
    # 1st options without image augmentation and dropout layer
    # 2nd options with image augmentation without dropout layer
    # 3rd options with image augmentation and dropout layer
    train_generator, validation_generator \
    = do_data_preprocessing(train_dir, validation_dir, aug=True)
    # Build a CNN model
    cnn_model = create_cnn_model()
    history = cnn_model.fit(
    train_generator,
    # steps_per_epoch=100, # 2000 images = batch_size * steps
```

```
    epochs=100,
    validation_data=validation_generator,
    # validation_steps=50,
    verbose=1
    )
    cnn_model.save("my_cat_dog_model.h5")
    acc = history.history["accuracy"]
    val_acc = history.history["val_accuracy"]
    loss = history.history["loss"]
    val_loss = history.history["val_loss"]
    plot_history(acc, val_acc, "Training and validation accuracy")
    plot_history(loss, val_loss, "Training and validation loss")
    # test the model
    fn_arr = ["anjing1.jpg”, "anjing2.jpg”, kucing1.jpg", "kucing2.jpg"]
    classify_images(fn_arr, cnn_model)
```

Hasil






```
Epoch 97/100
100/100 ━━━━━━━━━━━━━━━━━━━━ 77s 763ms/step -
accuracy: 0.8493 - loss: 0.3326 - val_accuracy:
0.8080 - val_loss: 0.4146
Epoch 98/100
100/100 ━━━━━━━━━━━━━━━━━━━━ 68s 678ms/step -
accuracy: 0.8623 - loss: 0.3447 - val_accuracy:
0.8140 - val_loss: 0.3946
Epoch 99/100
100/100 ━━━━━━━━━━━━━━━━━━━━ 69s 692ms/step -
accuracy: 0.8639 - loss: 0.3097 - val_accuracy:
0.7930 - val_loss: 0.4413
Epoch 100/100
100/100 ━━━━━━━━━━━━━━━━━━━━ 64s 634ms/step -
accuracy: 0.8443 - loss: 0.3459 - val_accuracy:
0.8110 - val_loss: 0.3930
WARNING:absl:You are saving your model as an HDF5
file via `model.save()` or
`keras.saving.save_model(model)`. This file format
is considered legacy. We recommend using instead
the native Keras format, e.g.
`model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.
1/1 ━━━━━━━━━━━━━━━━━━━━ 0.50s 1/4 step
[0.62]
anjing1.jpg is predicted as: dog

1/1 ━━━━━━━━━━━━━━━━━━━━ 0.12s 2/4 step
[0.97]
anjing2.jpg is predicted as: dog

1/1 ━━━━━━━━━━━━━━━━━━━━ 0.16s 3/4 step
[0.41]
kucing1.jpg is predicted as: cat

1/1 ━━━━━━━━━━━━━━━━━━━━ 0.18s 4/4 step
[0.15]
kucing2.jpg is predicted as: cat
```

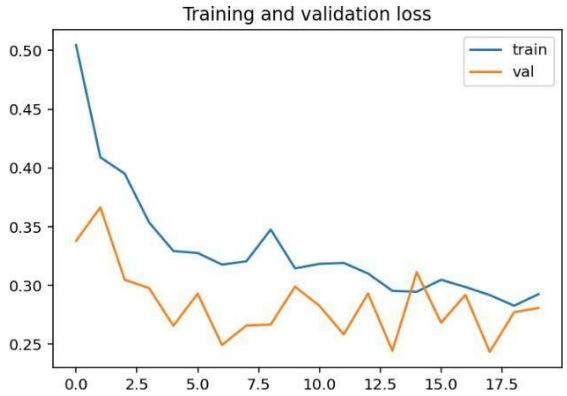
Cats and dogs classifier with transfer learning

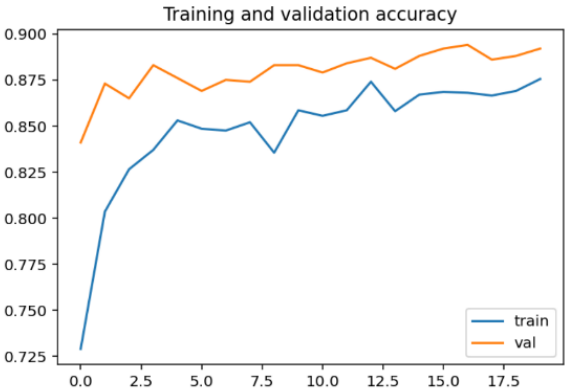
```
import os
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras import layers
from tensorflow.keras import Model
from tensorflow.keras.applications.vgg16 import VGG16
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.preprocessing import image as keras_image
def plot_history(train, val, title):
    epochs = range(len(train))
    plt.figure()
    plt.plot(epochs, train, label="train")
    plt.plot(epochs, val, label="val")
    plt.title(title)
    plt.legend(loc="best")
    plt.pause(1.0)
def do_data_preprocessing(train_dir, validation_dir, aug=False):
    if aug:
        train_datagen = ImageDataGenerator(
        rescale=1./255,
        rotation_range=40,
        width_shift_range=0.2,
        height_shift_range=0.2,
        shear_range=0.2,
        zoom_range=0.2,
        horizontal_flip=True,
        fill_mode="nearest"
        )
    else:
        train_datagen = ImageDataGenerator(rescale=1./255)
    validation_datagen = ImageDataGenerator(rescale=1. / 255)
    train_generator = train_datagen.flow_from_directory(
    train_dir,
    batch_size=20,
    class_mode="binary",
    target_size=(150, 150)
    )
    validation_generator = validation_datagen.flow_from_directory(
    validation_dir,
    batch_size=20,
    class_mode="binary",
    target_size=(150, 150)
    )
    return train_generator, validation_generator
def create_cnn_model(local_weights_file):
    pre_trained_model = VGG16(input_shape=(150, 150, 3),include_top=False,
```

```
    weights=None)
    pre_trained_model.load_weights(local_weights_file)
    for layer in pre_trained_model.layers:
        layer.trainable = False
    pre_trained_model.summary()
    last_layer = pre_trained_model.get_layer("block5_pool")
    print("last layer output shape: ", last_layer.output_shape)
    last_output = last_layer.output
    x = layers.Dropout(0.2)(last_output)
    x = layers.Flatten()(x)
    x = layers.Dense(1024, activation="relu")(x)
    x = layers.Dense(1, activation="sigmoid")(x)
    model = Model(pre_trained_model.input, x)
    model.compile(optimizer=Adam(learning_rate=1e-4),
    loss="binary_crossentropy",
    metrics=["accuracy"])
    return model
def classify_images(fn_arr, model):
    for fn in fn_arr:
        path = "datasets/" + fn
        img = keras_image.load_img(path, target_size=(150, 150))
        x = keras_image.img_to_array(img)
        x = np.expand_dims(x, axis=0)
        image_i = np.vstack([x])
        classes = model.predict(image_i, batch_size=10)
        print(classes[0])
        if classes[0] > 0.5:
            print(fn + " is a dog")
        else:
            print(fn + " is a cat")

if __name__ == "__main__":
    local_weights_file = "pre-
trained/vgg16_weights_tf_dim_ordering_tf_kernels_notop.h5"
    base_dir = "datasets/cats_and_dogs_filtered"
    train_dir = os.path.join(base_dir, "train")
    validation_dir = os.path.join(base_dir, "validation")
    train_cats_dir = os.path.join(train_dir, "cats")
    train_dogs_dir = os.path.join(train_dir, "dogs")
    validation_cats_dir = os.path.join(validation_dir, "cats")
    validation_dogs_dir = os.path.join(validation_dir, "dogs")
    train_cat_fnames = os.listdir(train_cats_dir)
    train_dog_fnames = os.listdir(train_dogs_dir)
    print(len(train_cat_fnames))
    print(len(train_dog_fnames))
    train_generator, validation_generator \
    = do_data_preprocessing(train_dir, validation_dir, aug=True)
    # Build a CNN model with pre-trained VGG16 net
    cnn_model = create_cnn_model(local_weights_file)
    history = cnn_model.fit(
    train_generator,
    #steps_per_epoch=100,
    epochs=20,
    validation_data=validation_generator,
```

```
    #validation_steps=50,
    verbose=1
    )
    acc = history.history["accuracy"]
    val_acc = history.history["val_accuracy"]
    loss = history.history["loss"]
    val_loss = history.history["val_loss"]
    plot_history(acc, val_acc, "Training and validation accuracy")
    plot_history(loss, val_loss, "Training and validation loss")
    # test the model
    fn_arr = ["cat-2083492_only_head.jpg", "cat-1146504_640.jpg",
    "dog-3846767_640.jpg", "dog-3388069_640.jpg"]
    classify_images(fn_arr, cnn_model)
```

Hasil






```
[0.]
cat-2083492_only_head.jpg is a cat
[0.]
cat-1146504_640.jpg is a cat
[1.]
dog-3846767_640.jpg is a dog
[1.]
dog-3388069_640.jpg is a dog
```

Hand sign language classifier

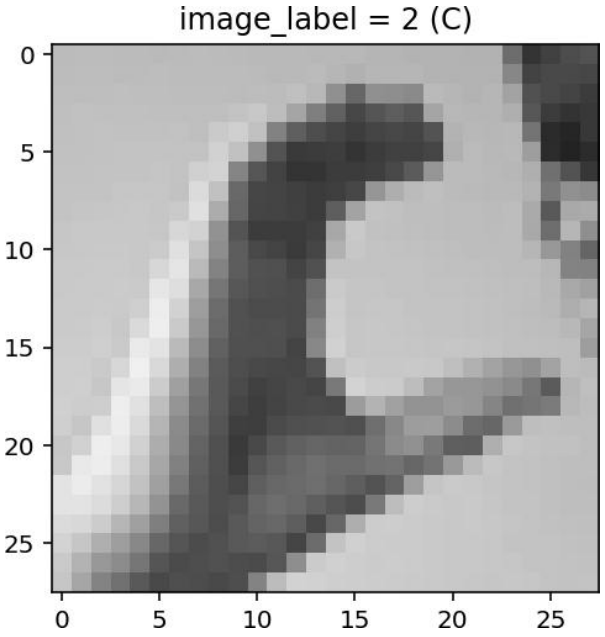
```
import os
import sys
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.preprocessing import image as keras_image
import tensorflow as tf
def get_data(filename):
    with open(filename) as training_file:
        _ = training_file.readline() # skip first line
        data = training_file.readlines()
        labels = []
        images = []
        num_of_data = len(data)
        for i, row in enumerate(data):
            row = row.strip("\n").split(",")
            labels.append(row[0])
            images.append(np.array_split(row[1:785], 28))
            sys.stdout.write(f"\rprocessing: {(i + 1) / float(num_of_data) *
100:.2f} %")
            sys.stdout.flush()
        print("")
        labels = np.array(labels).astype(float)
        images = np.array(images).astype(float)
    return images, labels

def plot_one_image(image_data, image_label):
    fig, ax = plt.subplots()
    ax.imshow(image_data, cmap="gray", vmin=0, vmax=255)
    num_to_alphabet = ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I',
    'K', 'L', 'M', 'N', 'O', 'P', 'Q', 'R', 'S',
    'T', 'U', 'V', 'W', 'X', 'Y']
    ax.set_title("image_label = {:g} ({:s})".format(image_label,
    num_to_alphabet[int(image_label)]))
    plt.pause(1.0)
def do_data_preprocessing(training_images, training_labels,
    validation_images, validation_labels):
    train_datagen = ImageDataGenerator(
    rescale=1. / 255,
    rotation_range=40,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.2,
    zoom_range=0.2,
    fill_mode="nearest"
    )
    validation_datagen = ImageDataGenerator(rescale=1. / 255)
```

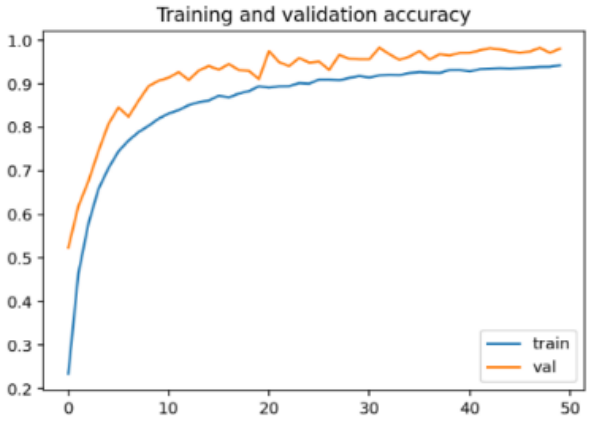
```
    training_generator = train_datagen.flow(
    training_images,
    training_labels,
    batch_size=20,
    )
    validation_generator = validation_datagen.flow(
    validation_images,
    validation_labels,
    batch_size=20
    )
    return training_generator, validation_generator
def create_cnn_model():
    # image size is 28x28, we don't need third conv2d
    # This will make the image size 1x1px!
    # second conv2D will make the image size 5x5px
    model = tf.keras.models.Sequential([
    tf.keras.layers.Conv2D(64, (3, 3), activation="relu", input_shape=(28, 28, 1)),
    tf.keras.layers.MaxPooling2D(2, 2),
    # tf.keras.layers.Conv2D(64, (3, 3), activation="relu"), #
tf.keras.layers.MaxPooling2D(2, 2),
    # tf.keras.layers.Conv2D(128, (3, 3), activation="relu"),
    # tf.keras.layers.MaxPooling2D(2, 2),
    tf.keras.layers.Dropout(0.1),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(1024, activation="relu"),
    tf.keras.layers.Dense(26, activation="softmax") # labels have value 0 - 24
    ])
    model.compile(
    loss="sparse_categorical_crossentropy",
    optimizer=Adam(learning_rate=0.001),
    metrics=["accuracy"]
    )
    model.summary()
    return model

def plot_history(train, val, title):
    epochs = range(len(train))
    plt.figure()
    plt.plot(epochs, train, label="train")
    plt.plot(epochs, val, label="val")
    plt.title(title)
    plt.legend(loc="best")
    plt.pause(1.0)
def classify_images(fn_arr, model):
    for fn in fn_arr:
        path = "datasets/" + fn
        # because we train using grayscale image, we need to convert
        # the sample image using color_mode="grayscale"
        img = keras_image.load_img(path, target_size=(28, 28),
        color_mode="grayscale")
        x = keras_image.img_to_array(img)
        x = np.expand_dims(x, axis=0)
        num_to_alphabet = ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I',
```

```
        'J', 'K', 'L', 'M', 'N', 'O', 'P', 'Q', 'R',
        'S', 'T', 'U', 'V', 'W', 'X', 'Y', 'Z']
        num_to_alphabet = np.array(num_to_alphabet)
        image_i = np.vstack([x])
        classes = model.predict(image_i, batch_size=10)
        print(classes[0])
        class_label = num_to_alphabet[classes[0].astype(np.int32) > 0.5][0]
        print(fn + " is a letter {:s}".format(class_label))

if __name__ == "__main__":
    # Load the dataset
    training_images, training_labels \
    = get_data(os.getcwd() + "/datasets/sign_mnist_train.csv")
    validation_images, validation_labels \
    = get_data(os.getcwd() + "/datasets/sign_mnist_test.csv")
    print(training_images.shape)
    print(training_labels.shape)
    print(validation_images.shape)
    print(validation_labels.shape)
    training_images = np.expand_dims(training_images, axis=-1)
    validation_images = np.expand_dims(validation_images, axis=-1)
    print(training_images.shape)
    print(validation_images.shape)
    print(np.max(validation_labels), np.min(validation_labels))
    print(np.max(training_labels), np.min(training_labels))
    # Plot one of the images and its label
    image_num = 2
    plot_one_image(training_images[image_num, :, :, 0],
    training_labels[image_num])
    # Data pre-preprocessing with ImageDataGenerator
    training_generator, validation_generator \
    = do_data_preprocessing(training_images, training_labels,
    validation_images, validation_labels)
    # Build a CNN model
    cnn_model = create_cnn_model()
    history = cnn_model.fit(
    training_generator,
    # steps_per_epoch=len(training_images)/20,
    epochs=50,
    validation_data=validation_generator,
    # validation_steps=len(validation_images)/20
    )
    #Evaluate the model on the validation dataset
    cnn_model.evaluate(validation_generator)
    acc = history.history["accuracy"]
    val_acc = history.history["val_accuracy"]
    loss = history.history["loss"]
    val_loss = history.history["val_loss"]
    plot_history(acc, val_acc, "Training and validation accuracy")
    plot_history(loss, val_loss, "Training and validation loss")
    # Test on sample images
    fn_arr = ["alphabet-letter-C-1298289_640.png",
    "alphabet-letter-D-1298315_640.png",
```

```
    "alphabet-letter-Y-1298311_640.png",
    "sign-language-letter-A-28717_640.png"]
    classify_images(fn_arr, cnn_model)
```

Hasil





```
Epoch 50/50
1373/1373 [==============================] - 74s 54ms/step - loss: 0.1828 - accuracy: 0.9419 -
val_loss: 0.0715 - val_accuracy: 0.9802
359/359 [==============================] - 3s 9ms/step - loss: 0.0715 - accuracy: 0.9802
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0.]
alphabet-letter-C-1298289_640.png is a letter P
[0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0.]
alphabet-letter-D-1298315_640.png is a letter I
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0.]
alphabet-letter-Y-1298311_640.png is a letter N
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0.]
sign-language-letter-A-28717_640.png is a letter N
```

D. ANALISA

Pada praktikum ini, dilakukan evaluasi kemampuan Convolutional Neural Network (CNN) dalam berbagai tugas klasifikasi citra. Pertama, model dibangun untuk dataset Fashion MNIST dengan dua lapisan konvolusi berjumlah 64 filter dan fungsi aktivasi ReLU, diikuti oleh MaxPooling, Flatten, dan dua lapisan Dense berakhir pada softmax untuk 10 kelas. Setelah 10 epoch pelatihan, akurasi training mencapai sekitar 96%, sedangkan pada data uji model berhasil meraih akurasi 91,8%, sehingga CNN efektif mengekstrak pola sederhana pada gambar fashion.

Selanjutnya, pada program klasifikasi ekspresi wajah memanfaatkan dataset emoji dengan dua kategori (happy dan sad). Arsitektur CNN serupa namun dengan output sigmoid dan callback penghenti otomatis saat akurasi melampaui 99%. Model mencapai 100% akurasi hanya dalam enam epoch, namun ketika diuji menggunakan foto wajah manusia, performanya menurun karena perbedaan karakteristik data, sehingga menunjukkan keterbatasan generalisasi bila data latih tidak representatif.

Pada program klasifikasi kucing dan anjing tanpa teknik khusus, model ini menunjukkan overfitting parah: akurasi pelatihan hampir 100% sementara validasi stagnan di sekitar 72%, dengan loss validasi yang justru meningkat. Untuk mengatasinya, perlu menambahkan augmentasi citra dan lapisan dropout 0,5. Hasilnya berhasil menstabilkan kurva akurasi dan loss, sehingga akurasi validasi meningkat menjadi sekitar 80%, serta menghasilkan tiga prediksi benar dari empat gambar uji.

Selanjutnya pada program memanfaatkan transfer learning menggunakan VGG16 pretrained tanpa fully connected layer atas. Dengan menambahkan augmentasi data dan dropout 0,2 setelah lapisan konvolusi terakhir, model ini mampu mencapai akurasi validasi sebesar 89,02%. Selain itu, performa validasi konsisten lebih tinggi daripada training, sehingga terdapat peningkatan generalisasi berkat arsitektur pra-terlatih dan regularisasi.

Kemudian yang terakhir, pada program klasifikasi bahasa isyarat menggunakan dataset Sign Language MNIST (28×28 piksel, 26 kelas). Menggunakan model sederhana dengan satu lapisan Conv2D, MaxPooling, Dropout, dan dua Dense layer mampu mencapai akurasi validasi hingga 98%, tetapi mengalami kegagalan saat diuji pada gambar eksternal.

E. KESIMPULAN

Convolutional Neural Network (CNN) mampu mengekstrak fitur visual dasar dengan sangat efektif pada berbagai tugas klasifikasi, tetapi performanya sangat bergantung pada kualitas dan keberagaman data serta teknik regularisasi. Pada dataset Fashion MNIST, CNN mencapai akurasi uji 91,8%, namun mengalami kesulitan pada klasifikasi ekspresi wajah saat data latih dan uji berbeda karakteristik. Untuk membedakan kucing dan anjing, penggunaan augmentasi dan dropout meningkatkan akurasi validasi dari 72% menjadi sekitar 80%, dan transfer learning dengan VGG16 lebih lanjut mengangkat akurasi menjadi 89,0%. Sementara itu, model sederhana pada klasifikasi bahasa isyarat Sign Language MNIST mencapai akurasi validasi 98% namun masih rentan terhadap data eksternal.